PRPrompt: Prototype-Routed Graph Prompting for Few-Shot Node Classification

Abstract

We introduce PRPrompt, a graph prompting method that uses support-set prototypes to route trainable prompts to individual nodes. With only a few labeled nodes per class, adaptation must determine both what a prompt should encode and where its contribution should be applied. PRPrompt constructs class references from aggregated node attributes, refines them using unlabeled graph features while retaining support anchors, and converts node–prototype associations into a fixed routing matrix. The resulting combinations of class prompt tokens are injected into a frozen graph convolutional encoder, followed by prototype classification. We evaluate each 1-, 3-, and 5-shot setting separately on Cora, CiteSeer, and PubMed using matched support draws and training seeds. In the 5-shot setting, PRPrompt achieves accuracies of 73.02%, 66.17%, and 70.98%, respectively; replacing a shared prompt with prototype routing improves accuracy by 36.94, 40.45, and 3.02 percentage points in the corresponding conditions. These paired results support the value of prototype-guided prompt allocation within the evaluated transductive setting.

1 Introduction

Graph prompting adapts a pretrained graph neural network using a limited set of task-specific parameters. This is attractive for few-shot node classification, where only a handful of labels are available for each class. Existing approaches adapt the input feature space, introduce prompt structures, or modify the readout of a frozen representation [1, 3, 7]. Alongside the choice of where to inject a prompt, a basic design decision concerns how to distribute its contribution across nodes.

A shared prompt applies the same modification to every node at the injection point. It can express a global adjustment, but its allocation does not depend on the relation between a node and the target classes. Input-dependent prompt combinations provide a more flexible alternative [1]. In a few-shot task, however, the small labeled support set also supplies an explicit set of class references. We ask whether these references can organize prompt allocation directly, so that learning focuses on prompt content rather than on an additional routing network.

Our hypothesis is that support-set prototypes can serve as coordinates for node-specific adaptation. Nodes related to a similar task class can receive related combinations of prompts, while different class relationships can induce different adjustments. The prototypes need not be learned model parameters: they can be computed from graph attributes and the support labels before prompt optimization. This separates the computation of task-conditioned allocation from the optimization of prompt tokens.

We instantiate this idea as PRPrompt. The method first forms centered, normalized representations of two-hop aggregated attributes. Support examples initialize class prototypes, which are refined using unlabeled node features with an explicit anchor to the support initialization. Node–prototype associations are centered across the graph and aggregated over neighbors to obtain a fixed routing matrix. This matrix mixes a small bank of trainable class prompts, which enter the first hidden layer of a frozen GCN. Prediction uses a prototype readout. Figure 1 identifies the roles of the fixed routing and trainable prompts.

We evaluate adaptation and routing with complementary comparisons. The main comparison includes a frozen pretrained encoder, full fine-tuning, and three prompt operators implemented under a common node-classification protocol. The routing ablation holds the prompt parameterization, injection point, optimization recipe, and readout fixed, replacing node-dependent routing with a shared mixture. A no-prompt condition measures the effect of the complete adaptation pipeline. All results are reported separately for Cora, CiteSeer, and PubMed at each shot count; averaging is restricted to seeds within a fixed condition.

Our contributions are:

- A prototype-routed prompt construction that turns support-conditioned class relationships into fixed node-specific coefficients for adapting a frozen graph encoder.
- A compact implementation with $C \times 128$ trainable prompt parameters, without a learned routing network, and an explicit separation between unlabeled feature use and label-based fitting.
- Matched comparisons and routing ablations for three citation datasets at 1, 3, and 5 shots, with condition-specific seed variability and paired uncertainty intervals.

The contribution concerns the use of class references to organize hidden-layer prompts. It does not assert that prototypes, unlabeled prototype refinement, or node-dependent prompting are individually new.

2 Related Work

Graph prompting and task adaptation. GPPT studies graph pretraining and prompt tuning for downstream adaptation [6]. GraphPrompt organizes pretraining and downstream prediction through a common similarity-based template and adapts task-specific readout [3]. All in One introduces prompt tokens, their structure, and insertion patterns within a multi-task graph prompting framework [7]. GPF and GPF-plus operate in the input feature space, including node-dependent prompt combinations [1]. Our distinction is the use of support-anchored prototype relationships to compute a fixed allocation of hidden-layer prompts. We do not equate node dependence itself with novelty or claim universal applicability across pretraining objectives.

Prototypes and unlabeled examples. Prototypical Networks classify examples through distances to support-set class representatives [5]. Semi-supervised few-shot learning has also used unlabeled examples to improve prototype construction [4]. These precedents motivate a restrained interpretation of our design: prototypes provide task references, while the contribution studied here is their role in routing prompt updates. The routing prototypes reside in aggregated attribute space and remain fixed during prompt training; the final prediction prototypes are constructed from the adapted encoder output. We evaluate known classes on one graph rather than meta-learning transfer to unseen classes.

Evaluation protocols. ProG provides a common framework and library for comparing graph prompting methods [8]. We use a ProG-compatible GCN and fixed edge-prediction checkpoints, and expose the differences between original methods and our common-protocol implementations. The resulting comparisons characterize this controlled setting; they are not reproductions of every baseline's strongest published configuration.

3 Problem Setting

Let $G = (V, E, X)$ be an attributed graph with n nodes, features $X \in \mathbb{R}^{n \times d}$, and C task classes. For each class c, a support set S_c contains k labeled nodes, with $k \in \{1, 3, 5\}$. Let $S = \bigcup_c S_c$ and let T be a disjoint query set. The full graph topology and all node features are visible; only labels in S are provided to fitting and hyperparameter selection. Query labels are used for scoring after predictions are sealed.

The encoder is a pretrained two-layer GCN [2] with hidden and output dimensions of 128. Its parameters are frozen for PRPrompt. The task is transductive, same-graph classification over known classes: query features participate in graph aggregation, prototype refinement, and global centering. It is not an inductive unseen-class evaluation. Throughout the paper, a *condition* fixes the dataset, shot count, method, and pretrained checkpoint.

4 Prototype-Routed Graph Prompting

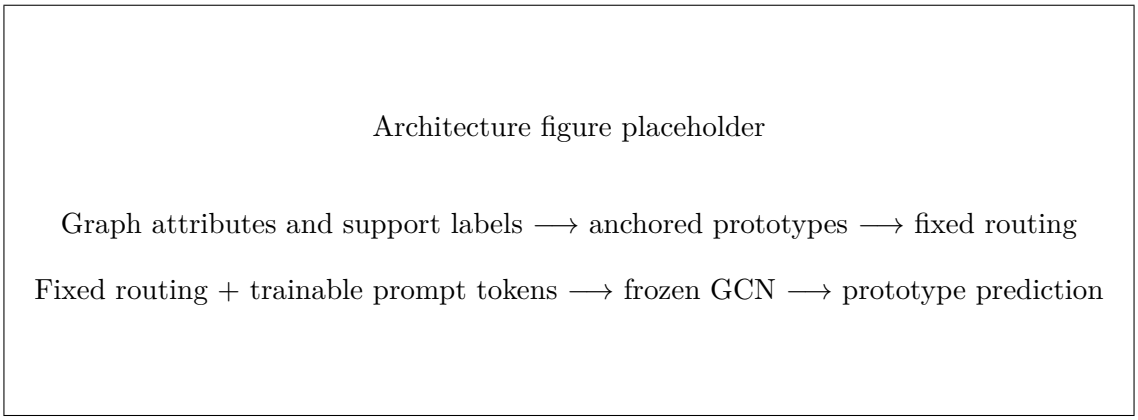


Figure 1: Overview of PRPrompt. Support labels initialize class references in aggregated attribute space. Unlabeled graph features refine these references, and the resulting node–prototype associations determine a fixed routing matrix. Only the class prompt tokens are optimized; the routed prompts enter the first hidden layer of a frozen GCN. Query labels are excluded from fitting and selection. The final architecture artwork is intentionally left as a placeholder.

4.1 Support-anchored class references

Let $\widehat{A} = \widetilde{D}^{-1/2}(A+I)\widetilde{D}^{-1/2}$ denote the normalized adjacency with self-loops. We form a routing representation from two-hop aggregated attributes:

$$R = \mathrm{rownorm}\Big(\widehat{A}^2 X - \mathbf{1}\ \mathrm{mean}_{v \in V}(\widehat{A}^2 X)_v\Big). \tag{1}$$

Here rownorm applies Euclidean normalization to each row. Routing therefore uses graph attributes and structure, rather than a separate learned network. Initial class references are

$$\mu_c^{(0)} = \frac{1}{k} \sum_{i \in S_c} R_i. \tag{2}$$

To incorporate the unlabeled graph while retaining a labeled reference, let $U_V = V \setminus S$.

Starting from $\mu^{(0)}$, we apply three fixed refinement iterations:

$$a_{ic}^{(t)} = \frac{\exp(R_i^\top \operatorname{norm}\left(\mu_c^{(t)}\right)/0.05)}{\sum_{c'} \exp(R_i^\top \operatorname{norm}\left(\mu_{c'}^{(t)}\right)/0.05)}, \quad i \in U_V, \tag{3}$$

$$\mu_c^{(t+1)} = 0.5\mu_c^{(0)} + 0.5\frac{\sum_{i\in U_V} a_{ic}^{(t)} R_i}{\max(\sum_{i\in U_V} a_{ic}^{(t)}, 10^{-8})}, \quad t = 0, 1, 2. \tag{4}$$

The anchor is always the original support prototype, not the previous iterate. This is a fixed prototype construction; we do not claim an expectation-maximization objective or a convergence guarantee. No labels from U_V enter these computations.

4.2 From class relationships to a fixed routing matrix

Using the refined prototypes, we compute node–class associations at temperature $\tau_r = 0.1$:

$$Q_{ic} = \operatorname{softmax}_c\left(R_i^\top \operatorname{norm}\left(\mu_c^{(3)}\right)/\tau_r\right), \qquad B = \widehat{A}\,(Q - \mathbf{1}\,\operatorname{mean}_{v\in V} Q_v)\,. \tag{5}$$

Centering expresses a node's class association relative to the graph-wide average; neighborhood aggregation incorporates adjacent association patterns. Q is a row-normalized association matrix, whereas B contains signed prompt-combination coefficients. We do not interpret B as a class-probability matrix. B is computed once per support set and kept fixed throughout training.

4.3 Adapting frozen hidden computations

Let H^{pre} be the first GCN layer's preactivation on the original graph. We learn a zero-initialized matrix $P \in \mathbb{R}^{C\times 128}$. With s the vector of channelwise sample standard deviations of H^{pre}, clamped below by 10^{-6}, the prompted hidden representation is

$$\widetilde{H}^{\text{pre}} = H^{\text{pre}} + 10\,[B\tanh(P)] \odot s. \tag{6}$$

The scale vector is broadcast over nodes. The existing LeakyReLU and second graph convolution then produce Z. Thus, prompt content is learned through support supervision, while routing and backbone parameters remain fixed. Zero initialization recovers the frozen encoder at the start of optimization. The trainable prompt counts are 896, 768, and 384 for Cora, CiteSeer, and PubMed.

4.4 Training and prediction

We optimize the prompt matrix for 100 Adam steps with learning rate 0.01 and no weight decay:

$$\mathcal{L}(P) = \frac{1}{|S|}\sum_{i\in S} \operatorname{CE}(\ell_i(P), y_i) + 10^{-3}\operatorname{mean}(P \odot P). \tag{7}$$

The support logits ℓ_i use cosine prototype classification at temperature 0.2. For 3 and 5 shots, a support node is excluded from its own class prototype when forming its training logits. At 1 shot, training uses the inclusive prototype because excluding the only labeled example would leave that class undefined. Prediction uses support prototypes after subtracting the global output mean and normalizing each node representation. These readout details are fixed throughout the routing ablation; Appendix A specifies them for reproduction.

The routing is a one-time computation. For sparse graphs, the two attribute aggregation steps cost $O(|E|d)$; prototype refinement costs $O(nCd)$ per iteration; and multiplying B by

prompt tokens costs $O(nC \cdot 128)$ per optimization step, in addition to the frozen encoder's forward and backward computations. Freezing weights reduces trainable parameters but does not eliminate gradient computation through the second convolution. We report parameter counts, not an unmeasured runtime advantage.

5 Experiments

5.1 Protocol and comparison methods

We evaluate Cora, CiteSeer, and PubMed at 1, 3, and 5 shots per class. The official training masks define the support pools and the official test masks define fixed query sets. Within a support draw, a shared per-class permutation yields nested 1-, 3-, and 5-shot sets. Each condition contains 20 support seeds and three training seeds. All methods share graph, checkpoint, and support/query identities within a condition. The pretrained source is the original edge-prediction checkpoint used by the project for each dataset; its pretraining seed is unavailable.

The main comparison contains: **Frozen**, with no prompt and the same final prototype readout as PRPrompt; **Full FT**, updating the pretrained encoder and a task readout; **GPF** and **GPF-plus**, adapting the input features; and **GraphPrompt**†, a diagonal readout-prompt implementation in the common node protocol. GPF-plus uses 20 prompt tokens. The dagger denotes a protocol adaptation, not a full reproduction of GraphPrompt's original subgraph-based pipeline. These prompt baselines are inspired by their original definitions [1, 3] but evaluated with the present backbone and selection rules.

PRPrompt and the shared-prompt ablation use one fixed recipe. For comparison methods that require selection, 3- and 5-shot fitting uses three support-only folds and four candidates: learning rate in $\{0.001, 0.01\}$ and weight decay in $\{0, 0.0001\}$. Each candidate fit and final fit uses 100 steps; 1-shot uses fixed parameters. Consequently, the methods do not have identical total optimization budgets. Appendix A gives the remaining settings.

Statistical unit. For a fixed condition, we first average three training seeds within each support draw, then report the mean and sample standard deviation over the 20 support draws. We never average performance across datasets or shot counts. Paired intervals resample support draws, preserving method pairing, with 10,000 bootstrap samples. They are unadjusted 95% intervals for the fixed graph and checkpoint. Accuracy and Macro-F1 are the primary reported metrics; the remaining metrics appear in Appendix B.

5.2 Adaptation relative to pretraining, fine-tuning, and prompting

This comparison tests whether the complete prototype-routed adaptation improves prediction under the same graph, checkpoint, and labeled support budget. Tables 1 and 2 report every condition, without a pooled average.

On Cora at 5 shots, PRPrompt reaches 73.02% accuracy compared with 33.56% for Frozen and 67.50% for Full FT. On CiteSeer at the same shot count, the corresponding accuracies are 66.17%, 27.38%, and 47.89%. PubMed has a stronger frozen reference: 68.37% at 5 shots, compared with 70.98% for PRPrompt and 63.96% for Full FT. Thus, the improvement over frozen pretraining is substantially smaller on PubMed, an important dependence that a cross-dataset average would hide.

The same condition-wise comparison favors PRPrompt over each of the three prompt operators in Accuracy and Macro-F1. All reported Accuracy and Macro-F1 paired intervals against the five main references are positive. These intervals quantify seed-dependent variability within

Table 1: Accuracy (%, higher is better). Main comparison. Within each dataset/shot/method, training seeds are averaged within each support draw; mean and sample standard deviation are then computed over 20 support draws. Bold marks the largest mean in that row. †: common-protocol adaptation.

Dataset / shot	Frozen	Full FT	GPF	GPF-plus	GraphPrompt†	PRPrompt
Cora / 1	23.17 ± 5.48	45.41 ± 10.18	15.20 ± 3.24	30.80 ± 6.64	21.88 ± 4.62	**54.59** ± 9.18
Cora / 3	29.42 ± 3.47	61.54 ± 6.78	24.32 ± 4.66	40.21 ± 4.80	27.75 ± 3.64	**70.76** ± 2.76
Cora / 5	33.56 ± 5.03	67.50 ± 5.06	28.17 ± 5.54	53.14 ± 5.55	31.36 ± 3.61	**73.02** ± 1.95
CiteSeer / 1	21.14 ± 2.87	35.76 ± 6.80	17.10 ± 2.64	21.40 ± 4.60	19.53 ± 3.34	**47.12** ± 6.31
CiteSeer / 3	25.29 ± 2.16	42.22 ± 5.88	20.36 ± 2.02	28.92 ± 2.68	24.83 ± 2.77	**61.20** ± 4.98
CiteSeer / 5	27.38 ± 2.84	47.89 ± 4.85	21.40 ± 2.57	31.82 ± 2.60	27.66 ± 2.49	**66.17** ± 2.16
PubMed / 1	55.44 ± 11.42	48.77 ± 8.54	36.29 ± 5.79	41.12 ± 6.83	50.24 ± 12.95	**58.70** ± 9.78
PubMed / 3	65.55 ± 8.11	59.75 ± 7.97	51.28 ± 6.93	57.46 ± 4.96	62.47 ± 5.54	**68.66** ± 4.58
PubMed / 5	68.37 ± 3.41	63.96 ± 5.18	55.83 ± 4.93	61.37 ± 5.32	66.94 ± 4.09	**70.98** ± 2.46

Table 2: Macro-F1 (%, higher is better). Main comparison. Within each dataset/shot/method, training seeds are averaged within each support draw; mean and sample standard deviation are then computed over 20 support draws. Bold marks the largest mean in that row. †: common-protocol adaptation.

Dataset / shot	Frozen	Full FT	GPF	GPF-plus	GraphPrompt†	PRPrompt
Cora / 1	22.51 ± 4.82	44.38 ± 10.07	7.08 ± 2.07	27.08 ± 6.92	21.03 ± 4.36	**53.34** ± 8.94
Cora / 3	29.07 ± 3.47	60.94 ± 6.41	21.14 ± 6.64	39.31 ± 4.55	26.98 ± 3.41	**69.38** ± 2.91
Cora / 5	33.07 ± 4.47	66.99 ± 4.52	25.87 ± 6.19	51.82 ± 5.06	30.82 ± 3.43	**72.04** ± 1.77
CiteSeer / 1	20.39 ± 2.62	30.99 ± 6.30	6.78 ± 1.56	12.83 ± 4.10	18.00 ± 2.95	**44.26** ± 5.75
CiteSeer / 3	24.32 ± 1.87	38.43 ± 5.80	13.81 ± 5.45	26.16 ± 2.85	23.71 ± 2.39	**58.19** ± 4.67
CiteSeer / 5	26.45 ± 2.71	44.62 ± 4.98	15.27 ± 5.85	29.46 ± 2.64	26.72 ± 2.49	**63.03** ± 2.12
PubMed / 1	53.44 ± 10.49	44.52 ± 9.85	25.62 ± 6.56	34.89 ± 8.06	45.35 ± 10.95	**57.37** ± 10.07
PubMed / 3	63.19 ± 8.60	57.64 ± 9.26	46.88 ± 9.09	54.47 ± 4.51	59.93 ± 5.76	**68.21** ± 4.08
PubMed / 5	66.27 ± 3.55	62.53 ± 6.05	52.42 ± 6.87	58.79 ± 5.19	64.79 ± 4.02	**70.39** ± 2.47

this setup; they do not establish superiority over all graph prompting methods or over independent graph populations. Full paired differences are provided in Appendix C.

5.3 Does prototype routing add value beyond a shared prompt?

The central ablation replaces B in Eq. (6) by the constant matrix with entries $1/C$. This retains the prompt matrix, scale, injection point, optimization, and final readout, but removes node-dependent allocation. The constant matrix is used directly; centering it would erase the prompt. We also retain the no-prompt Frozen reference.

Table 3: Accuracy (%, higher is better). Prototype-routing ablation. Within each dataset/shot/method, training seeds are averaged within each support draw; mean and sample standard deviation are then computed over 20 support draws. Bold marks the largest mean in that row.

Dataset / shot	Frozen	Shared prompt	PRPrompt
Cora / 1	23.17 ± 5.48	22.62 ± 4.41	**54.59** ± 9.18
Cora / 3	29.42 ± 3.47	31.70 ± 4.95	**70.76** ± 2.76
Cora / 5	33.56 ± 5.03	36.08 ± 4.19	**73.02** ± 1.95
CiteSeer / 1	21.14 ± 2.87	20.72 ± 2.46	**47.12** ± 6.31
CiteSeer / 3	25.29 ± 2.16	24.01 ± 2.70	**61.20** ± 4.98
CiteSeer / 5	27.38 ± 2.84	25.73 ± 2.78	**66.17** ± 2.16
PubMed / 1	55.44 ± 11.42	53.84 ± 10.55	**58.70** ± 9.78
PubMed / 3	65.55 ± 8.11	62.96 ± 7.94	**68.66** ± 4.58
PubMed / 5	68.37 ± 3.41	67.95 ± 3.11	**70.98** ± 2.46

Table 4: Macro-F1 (%, higher is better). Prototype-routing ablation. Within each dataset/shot/method, training seeds are averaged within each support draw; mean and sample standard deviation are then computed over 20 support draws. Bold marks the largest mean in that row.

Dataset / shot	Frozen	Shared prompt	PRPrompt
Cora / 1	22.51 ± 4.82	21.65 ± 3.66	**53.34** ± 8.94
Cora / 3	29.07 ± 3.47	30.25 ± 4.85	**69.38** ± 2.91
Cora / 5	33.07 ± 4.47	34.89 ± 4.10	**72.04** ± 1.77
CiteSeer / 1	20.39 ± 2.62	19.57 ± 2.33	**44.26** ± 5.75
CiteSeer / 3	24.32 ± 1.87	22.67 ± 2.46	**58.19** ± 4.67
CiteSeer / 5	26.45 ± 2.71	24.55 ± 2.74	**63.03** ± 2.12
PubMed / 1	53.44 ± 10.49	51.93 ± 9.81	**57.37** ± 10.07
PubMed / 3	63.19 ± 8.60	60.94 ± 8.60	**68.21** ± 4.08
PubMed / 5	66.27 ± 3.55	66.29 ± 2.87	**70.39** ± 2.47

Tables 3 and 4 show that a shared prompt alone does not account for the gains. At 5 shots, routing improves accuracy over the shared prompt by 36.94 percentage points on Cora, 40.45 on CiteSeer, and 3.02 on PubMed. The respective paired 95% intervals are $[34.87, 39.06]$, $[39.06, 41.84]$, and $[1.39, 4.51]$ percentage points. The improvement is also positive in both primary metrics in each 1- and 3-shot condition.

This comparison supports the complete prototype-routing construction relative to a shared prompt. It does not isolate prototype refinement, centering, and graph aggregation as separate causes. A uniform mixture also reduces the effective degrees of freedom despite retaining the nominal prompt parameter count. We therefore do not interpret the comparison as a capacity-matched proof of a unique routing mechanism.

6 Limitations

The evaluation covers three citation graphs, one fixed pretrained checkpoint per graph, and same-graph known-class classification. Pretraining seeds are unknown, so variation across pretraining runs is not estimated. The reporting scope was chosen after the experiments were completed; the analysis is descriptive rather than a new independent confirmatory study. It cannot support claims about graph classification, unseen classes, or arbitrary pretraining sources.

The common-protocol prompt baselines and their restricted hyperparameter search need not attain their best published results. Baseline selection also incurs a different training budget from the fixed PRPrompt recipe. The routing ablation changes effective prompt capacity and does not distinguish every component of the routing construction. Finally, the method uses query features and graph-wide statistics; deployment without such transductive access requires separate evaluation.

7 Conclusion

PRPrompt uses support-anchored prototypes to organize the allocation of trainable hidden-layer prompts in a frozen graph encoder. In the evaluated Cora, CiteSeer, and PubMed settings, condition-wise comparisons and shared-prompt ablations support the value of this allocation. The evidence favors prototype routing as a practical adaptation design within the tested protocol, while leaving its generality across graph families and pretraining sources open.

Broader Impact

The study concerns public citation-network benchmarks and does not evaluate decisions about individuals. Errors can propagate through graph aggregation and unlabeled prototype refinement, particularly when the graph or class structure differs from the evaluated setting. Applications involving consequential decisions require domain-specific evaluation beyond these benchmarks.

References

[1] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. Universal Prompt Tuning for Graph Neural Networks. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 52464–52489. Curran Associates, Inc., 2023. doi: 10.52202/075280-2285. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a4a1ee071ce0fe63b83bce507c9dc4d7-Paper-Conference.pdf.

[2] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[3] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023*, WWW '23, pages 417–428. ACM, April 2023. doi: 10.1145/3543507.3583386. URL http://dx.doi.org/10.1145/3543507.3583386.

[4] Mengye Ren, Eleni Triantafillou, Sachin Ravi, Jake Snell, Kevin Swersky, Joshua B. Tenenbaum, Hugo Larochelle, and Richard S. Zemel. Meta-Learning for Semi-Supervised Few-Shot Classification, 2018. URL https://arxiv.org/abs/1803.00676.

[5] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical Networks for Few-shot Learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/cb8da6767461f2812ae4290eac7cbc42-Paper.pdf.

[6] Mingchen Sun, Kaixiong Zhou, Xin He, Ying Wang, and Xin Wang. GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 1717–1727. ACM, August 2022. doi: 10.1145/3534678.3539249. URL http://dx.doi.org/10.1145/3534678.3539249.

[7] Xiangguo Sun, Hong Cheng, Jia Li, Bo Liu, and Jihong Guan. All in One: Multi-Task Prompting for Graph Neural Networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pages 2120–2131. ACM, August 2023. doi: 10.1145/3580305.3599256. URL http://dx.doi.org/10.1145/3580305.3599256.

[8] Chenyi Zi, Haihong Zhao, Xiangguo Sun, Yiqing Lin, Hong Cheng, and Jia Li. ProG: A Graph Prompt Learning Benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 95406–95437. Curran Associates, Inc., 2024. doi: 10.52202/079017-3023. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad3e803a977f4279330c6ab7245937c6-Paper-Datasets_and_Benchmarks_Track.pdf.

A Implementation and Reproducibility

A.1 Graph preprocessing and fixed checkpoints

Table 5 records the dataset dimensions from the execution plan. Features use the dataset loader's row normalization. Support pools and query sets come from the official training and test masks; validation labels are not used for fitting or selection. All other nodes remain visible as unlabeled graph context. Each task contains all classes of its graph, with the requested number of support nodes per class.

Table 5: Dataset dimensions and trainable prompt counts from the recorded protocol.

Dataset	Nodes	Features	Classes	Prompt parameters
Cora	2708	1433	7	896
CiteSeer	3327	3703	6	768
PubMed	19717	500	3	384

The encoder consists of two GCNConv layers of width 128 with LeakyReLU between them. The first layer includes the pretrained affine transformation and graph aggregation; the prompt is added to its output before the nonlinearity. The second convolution has no additional activation in the evaluated encoder. The same checkpoint is used by every comparison method on a dataset. Checkpoint, graph, and split hashes are preserved in the accompanying result records. The recorded pretraining family is `Edgepred_Gprompt`; the pretraining random seed is unavailable.

A.2 Training and prediction details

The main text describes the active forward computation. For reproducibility, Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, learning rate 0.01, and weight decay zero. The prompt regularizer is the mean of squared token entries multiplied by 10^{-3}. It is separate from optimizer weight decay. The channel scale uses the sample standard deviation, with denominator $n - 1$. Euclidean normalization uses the implementation's 10^{-12} norm floor. No distillation or query-label objective is active.

For the support loss, let $\bar{z}_i = \mathrm{norm}(Z_i)$ and $m_c = \sum_{j \in S_c} \bar{z}_j$. At $k > 1$, the normalized prototype used for training support node i is

$$p_c^{(-i)} = \mathrm{norm}\left(\frac{m_c - \mathbb{I}[y_i = c]\bar{z}_i}{k - \mathbb{I}[y_i = c]} \right), \qquad \ell_{ic} = \bar{z}_i^\top p_c^{(-i)} / 0.2. \tag{8}$$

For $k = 1$, the inclusive class prototype $\mathrm{norm}(m_c/k)$ replaces $p_c^{(-i)}$. During prediction, define

$$z_i^* = \mathrm{norm}\left(Z_i - \frac{1}{n} \sum_{j \in V} Z_j \right), \quad p_c^* = \mathrm{norm}\left(\frac{1}{k} \sum_{i \in S_c} z_i^* \right), \quad \widehat{y}_i = \arg\max_c (z_i^*)^\top p_c^*. \tag{9}$$

The probability scores for all five metrics are obtained by applying softmax to cosine logits divided by 0.2. Training does not apply the output centering used at prediction. The readout is a fixed implementation choice, not an experimental factor in this paper.

Procedure. Given a graph, a fixed checkpoint, and support labels:

1. Normalize adjacency and compute Eq. (1) and the first-layer preactivation.
2. Initialize support prototypes with Eq. (2) and apply Eqs. (3)–(4) three times.
3. Compute the fixed route with Eq. (5); for the shared-prompt ablation, replace it by $B_{ic} = 1/C$.
4. Initialize $P = 0$ and cache the hidden-channel scales. Keep the encoder and route fixed.
5. For 100 steps, run the prompted encoder, form support logits, and update P using Eq. (7).
6. Compute final prototype scores using Eq. (9); seal predictions before scoring with query labels.

A.3 Baseline fitting and seed control

The support-draw seeds are 30093000 through 30093019 and training seeds are 0, 1, and 2. A draw uses a seeded permutation in each class's support pool, taking its first k entries. All methods receive the same support identities for each draw. The no-training Frozen method retains the same three seed slots for alignment, but those slots do not represent three independent fits.

The learnable comparison methods use support cross-entropy. Full FT and the input-prompt methods include a task readout, whereas the GraphPrompt adaptation uses its prompted representation with prototype prediction. GPF adds a shared feature vector; GPF-plus learns 20 feature prompts and an input-dependent mixture. Their final one-shot parameters are learning rate 0.01 and weight decay 0.0001. For 3 and 5 shots, each of three support-only folds holds out one node per class. Four candidate configurations are ranked by mean held-out support cross-entropy, then the selected method is refit on the complete support set. No external validation-label budget is used. This protocol controls labeled information and final step count, but not total selection compute or readout parameterization.

A.4 Condition-specific statistics

For a fixed dataset, shot count, checkpoint, and method, let a_{st} denote a metric with support seed s and training seed t. We compute

$$\bar{a}_s = \frac{1}{3}\sum_{t=1}^{3} a_{st}, \quad \widehat{a} = \frac{1}{20}\sum_{s=1}^{20} \bar{a}_s, \quad \widehat{\sigma} = \sqrt{\frac{1}{19}\sum_{s=1}^{20}(\bar{a}_s - \widehat{a})^2}. \tag{10}$$

Tables report $100\widehat{a} \pm 100\widehat{\sigma}$, not the standard error. For paired comparisons, $d_s = \bar{a}_s^{\mathrm{PRPrompt}} - \bar{a}_s^{\mathrm{reference}}$ is resampled as a support-draw block. The percentile bounds of 10,000 resampled means form the reported 95% interval. Existing intervals are reused without modification. No metric average or mean effect is computed across different datasets, shot counts, or methods.

Accuracy, Macro-F1, Micro-F1, macro one-vs-rest ROC-AUC, and macro average precision are retained in the data package. In this single-label multiclass setting, Micro-F1 coincides with Accuracy; it is included for completeness. The fixed query set is shared across support draws, so intervals concern support selection and training variation conditional on that graph, not independent samples of graphs.

A.5 Environment, provenance, and numerical qualification

The execution plan records Python 3.11.16, PyTorch 2.11.0+cu130, and CUDA 13.0 in the reused ProG environment. Per-unit records identify CUDA devices and fitting durations. An isolated total GPU-hour budget for the present reporting scope and a verified accelerator inventory are not provided by the extracted package; no efficiency comparison is inferred from these results.

The tables derive from the sealed three-dataset package. It contains 3,240 main-comparison units and 540 additional shared-prompt units; PRPrompt contributes 540 units. Source-metric checksums and paired graph/checkpoint/support/query identities were verified during extraction. The source numerical audits report exact readout reproduction from saved representations for all 3,780 selected units, passing gate/forward tolerances, and no changed query classes in fresh forwards. These are archived verification results, not newly rerun experiments, and passing a tolerance does not imply bitwise-identical CUDA arithmetic.

Source code and the table data are available in the accompanying repository at `https://github.com/Mes-sala/PRPrompt`. The source revision of the recorded experiments is `0328a2bb68f249a34c89557178b6233daf5ce20d`. The paper package includes table-generation and static-validation scripts, hashes, and bibliographic provenance. Model checkpoints and optimizer states remain on the experiment server and are excluded from the local paper package. This manuscript reports existing experiments; no new training or bootstrap analysis was performed for its preparation.

B Additional Condition-Wise Metrics

The following tables extend the main comparison with the remaining stored metrics under the same seed aggregation rule. Every row is a separate dataset/shot condition.

Table 6: Micro-F1 (%, higher is better). Main comparison; seed aggregation follows Appendix A.

Dataset / shot	Frozen	Full FT	GPF	GPF-plus	GraphPrompt†	PRPrompt
Cora / 1	23.17 ± 5.48	45.41 ± 10.18	15.20 ± 3.24	30.80 ± 6.64	21.88 ± 4.62	**54.59** ± 9.18
Cora / 3	29.42 ± 3.47	61.54 ± 6.78	24.32 ± 4.66	40.21 ± 4.80	27.75 ± 3.64	**70.76** ± 2.76
Cora / 5	33.56 ± 5.03	67.50 ± 5.06	28.17 ± 5.54	53.14 ± 5.55	31.36 ± 3.61	**73.02** ± 1.95
CiteSeer / 1	21.14 ± 2.87	35.76 ± 6.80	17.10 ± 2.64	21.40 ± 4.60	19.53 ± 3.34	**47.12** ± 6.31
CiteSeer / 3	25.29 ± 2.16	42.22 ± 5.88	20.36 ± 2.02	28.92 ± 2.68	24.83 ± 2.77	**61.20** ± 4.98
CiteSeer / 5	27.38 ± 2.84	47.89 ± 4.85	21.40 ± 2.57	31.82 ± 2.60	27.66 ± 2.49	**66.17** ± 2.16
PubMed / 1	55.44 ± 11.42	48.77 ± 8.54	36.29 ± 5.79	41.12 ± 6.83	50.24 ± 12.95	**58.70** ± 9.78
PubMed / 3	65.55 ± 8.11	59.75 ± 7.97	51.28 ± 6.93	57.46 ± 4.96	62.47 ± 5.54	**68.66** ± 4.58
PubMed / 5	68.37 ± 3.41	63.96 ± 5.18	55.83 ± 4.93	61.37 ± 5.32	66.94 ± 4.09	**70.98** ± 2.46

Table 7: Macro ROC-AUC (%, higher is better). Main comparison; seed aggregation follows Appendix A.

Dataset / shot	Frozen	Full FT	GPF	GPF-plus	GraphPrompt†	PRPrompt
Cora / 1	62.95 ± 4.93	81.83 ± 5.73	53.20 ± 2.50	74.73 ± 4.00	61.93 ± 4.85	**83.06** ± 5.42
Cora / 3	68.95 ± 2.99	89.94 ± 2.35	64.61 ± 4.25	78.13 ± 2.49	68.05 ± 3.01	**90.84** ± 1.97
Cora / 5	72.36 ± 2.85	**92.58** ± 1.26	68.65 ± 4.34	85.66 ± 2.71	71.01 ± 2.66	92.47 ± 1.11
CiteSeer / 1	56.11 ± 3.24	70.97 ± 4.63	53.08 ± 1.84	64.19 ± 4.07	54.39 ± 4.21	**74.71** ± 5.81
CiteSeer / 3	60.17 ± 2.06	76.04 ± 3.00	55.85 ± 1.91	65.37 ± 3.23	59.99 ± 2.60	**82.82** ± 3.08
CiteSeer / 5	62.69 ± 2.35	79.54 ± 2.22	57.93 ± 1.77	68.91 ± 2.38	62.67 ± 2.33	**85.86** ± 1.52
PubMed / 1	73.87 ± 11.03	70.73 ± 9.76	60.13 ± 8.31	68.63 ± 6.61	70.43 ± 11.06	**76.48** ± 10.43
PubMed / 3	82.17 ± 6.59	81.00 ± 3.74	72.19 ± 4.91	76.82 ± 4.24	81.13 ± 4.42	**86.29** ± 3.10
PubMed / 5	84.83 ± 2.59	84.31 ± 2.45	74.64 ± 4.10	79.98 ± 2.95	83.57 ± 2.81	**87.03** ± 1.87

Table 8: Macro AP (%, higher is better). Main comparison; seed aggregation follows Appendix A.

Dataset / shot	Frozen	Full FT	GPF	GPF-plus	GraphPrompt†	PRPrompt
Cora / 1	23.64 ± 4.26	54.01 ± 10.10	16.64 ± 1.47	40.37 ± 5.88	23.15 ± 3.89	**59.04** ± 9.07
Cora / 3	29.20 ± 3.71	70.20 ± 4.98	24.89 ± 3.11	44.43 ± 4.09	27.06 ± 2.89	**74.33** ± 3.38
Cora / 5	33.16 ± 4.26	75.99 ± 3.16	28.68 ± 3.71	58.84 ± 6.14	29.92 ± 3.11	**77.64** ± 1.87
CiteSeer / 1	21.53 ± 2.32	38.74 ± 5.84	19.66 ± 1.43	29.38 ± 4.20	20.61 ± 2.27	**45.99** ± 6.79
CiteSeer / 3	24.17 ± 1.59	45.56 ± 4.79	21.40 ± 1.15	30.59 ± 3.23	23.97 ± 1.89	**59.15** ± 4.80
CiteSeer / 5	26.46 ± 2.07	51.03 ± 4.40	22.88 ± 1.45	34.40 ± 2.70	26.04 ± 1.65	**64.02** ± 2.66
PubMed / 1	57.52 ± 10.93	55.36 ± 10.55	43.93 ± 6.89	52.61 ± 8.08	52.39 ± 9.87	**61.58** ± 11.39
PubMed / 3	67.27 ± 8.59	67.52 ± 6.82	55.29 ± 5.93	60.88 ± 5.58	65.03 ± 5.79	**74.22** ± 5.19
PubMed / 5	70.34 ± 4.13	72.33 ± 5.00	58.42 ± 5.12	65.42 ± 4.92	68.42 ± 3.67	**75.35** ± 3.34

C Paired Differences and Uncertainty

Each cell reports PRPrompt minus the named reference in percentage points, with its unadjusted 95% paired support-bootstrap interval. All entries preserve the original paired statistics. Tables group conditions for readability but do not average them.

Table 9: PRPrompt minus Frozen: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+31.42 [+28.20, +34.59]	+30.84 [+27.77, +33.84]
Cora / 3	+41.34 [+39.63, +43.04]	+40.32 [+38.64, +42.08]
Cora / 5	+39.46 [+37.23, +41.61]	+38.97 [+36.88, +40.99]
CiteSeer / 1	+25.98 [+23.72, +28.21]	+23.87 [+21.80, +25.77]
CiteSeer / 3	+35.91 [+33.05, +38.39]	+33.87 [+31.34, +36.14]
CiteSeer / 5	+38.78 [+37.37, +40.18]	+36.58 [+35.22, +37.95]
PubMed / 1	+3.26 [+0.40, +6.22]	+3.93 [+0.86, +7.00]
PubMed / 3	+3.11 [+0.38, +6.27]	+5.02 [+2.27, +8.44]
PubMed / 5	+2.61 [+1.13, +4.22]	+4.12 [+2.53, +5.91]

Table 10: PRPrompt minus Full FT: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+9.17 [+6.58, +11.87]	+8.97 [+6.40, +11.77]
Cora / 3	+9.22 [+6.75, +11.79]	+8.44 [+6.12, +10.90]
Cora / 5	+5.52 [+3.73, +7.50]	+5.05 [+3.34, +6.86]
CiteSeer / 1	+11.36 [+9.57, +13.18]	+13.27 [+11.17, +15.31]
CiteSeer / 3	+18.98 [+16.78, +20.94]	+19.76 [+17.45, +22.00]
CiteSeer / 5	+18.28 [+16.22, +20.13]	+18.42 [+16.07, +20.60]
PubMed / 1	+9.93 [+6.77, +13.14]	+12.86 [+9.39, +16.37]
PubMed / 3	+8.90 [+5.89, +12.26]	+10.57 [+7.08, +14.41]
PubMed / 5	+7.02 [+4.96, +9.14]	+7.86 [+5.54, +10.28]

Table 11: PRPrompt minus GPF: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+39.38 [+35.60, +42.98]	+46.27 [+42.36, +49.82]
Cora / 3	+46.44 [+44.21, +48.74]	+48.25 [+45.22, +51.53]
Cora / 5	+44.86 [+42.52, +47.57]	+46.17 [+43.71, +49.32]
CiteSeer / 1	+30.02 [+27.04, +33.00]	+37.48 [+34.93, +39.92]
CiteSeer / 3	+40.83 [+38.11, +43.06]	+44.38 [+40.82, +47.90]
CiteSeer / 5	+44.77 [+43.32, +46.16]	+47.77 [+44.95, +50.72]
PubMed / 1	+22.40 [+18.82, +26.24]	+31.76 [+28.23, +35.27]
PubMed / 3	+17.37 [+14.16, +21.02]	+21.33 [+17.31, +25.89]
PubMed / 5	+15.15 [+13.30, +17.01]	+17.97 [+15.46, +20.67]

Table 12: PRPrompt minus GPF-plus: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+23.78 [+19.96, +27.47]	+26.26 [+22.23, +30.20]
Cora / 3	+30.55 [+28.29, +32.52]	+30.07 [+28.04, +31.97]
Cora / 5	+19.88 [+17.62, +22.16]	+20.22 [+18.02, +22.39]
CiteSeer / 1	+25.71 [+22.89, +28.45]	+31.43 [+28.61, +34.12]
CiteSeer / 3	+32.27 [+30.45, +34.03]	+32.03 [+30.10, +33.81]
CiteSeer / 5	+34.35 [+33.05, +35.65]	+33.57 [+32.22, +34.87]
PubMed / 1	+17.57 [+13.28, +21.86]	+22.48 [+18.10, +26.98]
PubMed / 3	+11.20 [+9.47, +13.11]	+13.74 [+12.07, +15.56]
PubMed / 5	+9.61 [+7.84, +11.49]	+11.61 [+9.80, +13.53]

Table 13: PRPrompt minus GraphPrompt†: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+32.70 [+29.19, +36.29]	+32.32 [+28.93, +35.68]
Cora / 3	+43.01 [+41.38, +44.73]	+42.41 [+40.61, +44.20]
Cora / 5	+41.66 [+40.03, +43.20]	+41.22 [+39.64, +42.75]
CiteSeer / 1	+27.59 [+24.97, +30.09]	+26.26 [+23.90, +28.62]
CiteSeer / 3	+36.37 [+33.26, +38.82]	+34.48 [+31.80, +36.73]
CiteSeer / 5	+38.51 [+37.14, +39.90]	+36.31 [+34.99, +37.65]
PubMed / 1	+8.45 [+3.60, +13.47]	+12.02 [+7.33, +16.52]
PubMed / 3	+6.19 [+4.04, +8.30]	+8.27 [+6.21, +10.36]
PubMed / 5	+4.03 [+1.94, +6.36]	+5.60 [+3.49, +7.88]

Table 14: PRPrompt minus Shared prompt: paired differences and 95% intervals (percentage points).

Dataset / shot	Accuracy	Macro-F1
Cora / 1	+31.96 [+28.59, +35.21]	+31.70 [+28.29, +35.00]
Cora / 3	+39.06 [+36.49, +41.44]	+39.13 [+36.60, +41.48]
Cora / 5	+36.94 [+34.87, +39.06]	+37.15 [+35.09, +39.23]
CiteSeer / 1	+26.40 [+24.07, +28.73]	+24.69 [+22.70, +26.65]
CiteSeer / 3	+37.18 [+34.90, +39.33]	+35.52 [+33.31, +37.55]
CiteSeer / 5	+40.45 [+39.06, +41.84]	+38.48 [+37.22, +39.72]
PubMed / 1	+4.85 [+1.26, +8.26]	+5.44 [+1.84, +9.07]
PubMed / 3	+5.69 [+3.25, +8.27]	+7.27 [+4.64, +10.29]
PubMed / 5	+3.02 [+1.39, +4.51]	+4.10 [+2.56, +5.58]